

# Sistema Genérico de Autenticação

## Análise de Requisitos

### 1. Introdução

Este documento apresenta a Análise de Requisitos detalhada para o desenvolvimento do **Sistema Genérico de Autenticação**. O objetivo é definir de forma clara e inequívoca as necessidades e expectativas do sistema, servindo como base para as fases subsequentes de design, desenvolvimento e testes. O resultado final deve ser seguro, escalável, íntegro e pronto para produção.

### 2. Requisitos Funcionais (RFs)

Os Requisitos Funcionais descrevem o comportamento que o sistema deve apresentar em termos de funcionalidades e serviços que serão providos ao usuário.

#### 2.1. Autenticação e Gestão de Contas

Estes requisitos definem como os usuários acessam o sistema e gerenciam suas informações de conta.

- **RF 2.1.1. Cadastro Mínimo de Usuário:** O sistema deve permitir que um novo usuário se registre, fornecendo e-mail e senha. Validação: {email} Rate Limit: {5} Idempotência
- **RF 2.1.2. Login:** O sistema deve permitir que usuários registrados façam login usando seu e-mail e senha. Rate Limit: {5}
- **RF 2.1.3. Login Social (OAuth2):** Permitir o acesso via Google ou conta Gov.br (em análise).
- **RF 2.1.4. Recuperação de Senha:** O sistema deve permitir que o usuário solicite a recuperação de senha via e-mail.
- **RF 2.1.5. Logout:** O sistema deve fornecer uma funcionalidade segura de encerramento de sessão (logout).
- **RF 2.1.6. Cadastro de Dados Opcionais:** O sistema deve permitir o preenchimento de dados adicionais não obrigatórios. Validação
- **RF 2.1.7. Confirmação de E-mail:** O sistema deve enviar um código ou link de ativação para o e-mail cadastrado. O acesso às funcionalidades de escrita (salvar dados) só deve ser liberado após a validação.
- **RF 2.1.8. Aceite de Termos de Uso e Privacidade:** O sistema deve exigir que o usuário aceite os Termos de Uso e a Política de Privacidade (em conformidade

com a **LGPD**) no momento do cadastro inicial. [Atomicidade Log de Auditoria](#)  
[Versionamento de Termos](#)

- **RF 2.1.9. Exclusão de Conta:** O sistema deve permitir que o usuário solicite a exclusão de sua conta.
- **RF 2.1.10. Gestão de Sessões Ativas:** O usuário deve ser capaz de visualizar e encerrar sessões ativas em outros dispositivos (importante para segurança financeira). [Log de Auditoria](#)
- **RF 2.1.11. Bloqueio de Tentativas:** O sistema deve bloquear login após 5 tentativas mal-sucedidas, enviando e-mail para mudança de senha. [Log de Auditoria](#)
- **RF 2.1.12. Invalidação de Sessões ao Trocar Senha:** Caso o usuário altere a senha ou recupere via e-mail, o sistema deve encerrar todas as outras sessões ativas imediatamente por segurança. [Atomicidade Log de Auditoria](#)

### 3. Detalhes Técnicos (DTs)

Os Detalhes Técnicos especificam as diretrizes de arquitetura, infraestrutura e protocolos de segurança que regem a implementação das funcionalidades, garantindo a integridade tecnológica e o desempenho do sistema.

#### 3.1. Arquitetura e Stack do Sistema

Detalhes técnicos da especificação da Arquitetura do sistema e a Stack utilizada.

- **DT 3.1.1 Estilo Arquitetural - Híbrido Pragmático:** O sistema adota uma arquitetura híbrida que combina a **Layered Architecture** (camadas horizontais), o **Domain-Driven Design** (organização por contextos) e **Ports & Adapters** (isolamento via interfaces SPI). Cada pacote adota a convenção mais adequada ao seu nível de abstração, evitando complexidade acidental em fluxos simples.
- **DT 3.1.2 Pacote domain:** Organizado por subdomínios técnicos ou de negócio. Cada subdomínio é uma unidade independente que pode conter os seguintes subpacotes fixos, conforme a necessidade:
  - **entity:** Modelos de dados persistentes (JPA).
  - **dto:** Objetos de transferência para entrada e saída da API.
  - **enums:** Constantes e conceitos de domínio enumerados.
  - **pk:** Classes de chaves primárias compostas (se aplicável).
  - **util:** Lógicas utilitárias exclusivas daquele subdomínio.

*Nota: A ausência de uma necessidade específica (ex: chave composta) dispensa a criação do subpacote correspondente.*

- **DT 3.1.3 Pacote service:** Contém a orquestração da lógica de negócio e casos de uso. O subpacote `service/spi` define as **Portas de Saída** (interfaces de serviço); essas interfaces garantem que o domínio dite como as integrações devem se comportar, sem conhecer os detalhes da implementação.

- **DT 3.1.4 Pacote repository:** Atua como **Adaptador de Saída** para persistência, contendo as interfaces Spring Data JPA. Este pacote é deliberadamente separado do domínio para isolar a tecnologia de acesso a dados (infraestrutura) do modelo de negócio.
- **DT 3.1.5 Pacote controller:** Atua como **Adaptador de Entrada** para o protocolo HTTP. Os controllers possuem a responsabilidade única de receber requisições, delegar a execução para a camada de **service** e retornar respostas padronizadas via **ApiResponse<T>**.
- **DT 3.1.6 Pacote infrastructure:** Contém as implementações concretas das portas definidas em **service/spi** e configurações técnicas transversais. É subdividido por mecanismo ou tecnologia (ex: **infrastructure/security**, **infrastructure/queue**), servindo como o ponto de acoplamento com provedores externos e bibliotecas de terceiros.
- **DT 3.1.7 Pacote exception:** Utiliza exceções de domínio tipadas (ex: **ResourceNotFoundException**) lançadas pela camada de **service**. Um componente centralizador (**@ControllerAdvice**) intercepta essas falhas e as converte em respostas HTTP padronizadas, evitando vazamento de *stacktraces* para o cliente.
- **DT 3.1.8 Pacote response:** Todas as saídas da API devem ser encapsuladas no objeto **ApiResponse<T>**, garantindo que o cliente receba uma estrutura de JSON consistente (ex: campos **data**, **errors**, **timestamp**) em qualquer *endpoint* do sistema.
- **DT 3.1.9 Pacote AOP + annotation:** Funcionalidades que cortam múltiplas camadas (logging, segurança, auditoria) devem ser implementadas via **Spring AOP** ou anotações customizadas. Isso permite que a lógica de negócio permaneça "limpa" e focada apenas no domínio.
- **DT 3.1.10 Regra de Dependência:** O fluxo de dependência é estritamente unidirecional: **controller** → **service** → **repository/spi**. O pacote **domain** deve ser agnóstico a outras camadas. O pacote **infrastructure** implementa interfaces de **service/spi**, mas nunca expõe suas classes concretas para as camadas superiores.
- **DT 3.1.11 Linguagem e Versão:** O sistema é implementado em **Java 17 (LTS)**, utilizando recursos como *sealed classes*, *records*, *pattern matching for instanceof* e inferência de tipo local (*var*) onde aplicável.
- **DT 3.1.12 Framework Principal:** **Spring Boot 3.5.x** como framework base. O gerenciamento de versões de dependências Spring é feito via BOM herdado do *spring-boot-starter-parent*, sem declaração explícita de versão na maioria dos artefatos.
- **DT 3.1.13 Camada Web: Spring MVC** (*spring-boot-starter-web*) com servidor embutido Tomcat. APIs REST com serialização/deserialização via Jackson (*ObjectMapper*). Validação de entrada com Jakarta Bean Validation (*spring-boot-starter-validation*, anotações **@Valid**, **@NotNull**, **@Min**, etc.).
- **DT 3.1.14 Programação Orientada a Aspectos: Spring AOP** (*spring-boot-starter-aop*) para *concerns* transversais como logging de tempo

de execução. Aspectos são definidos com `@Aspect` e ativados via anotações customizadas com `@Retention(RUNTIME)`.

- **DT 3.1.15 Broker de Mensagens:** O sistema utiliza **RabbitMQ** como broker de mensagens assíncronas. A integração é feita via *spring-boot-starter-amqp*. O acoplamento é isolado via interface `QueuePublisher<T>` no pacote `service/spi`, de modo que a camada de serviço não depende diretamente da implementação AMQP.
- **DT 3.1.16 Padrão de Publicação:** Publicações são realizadas pela implementação concreta de `QueuePublisher<T>` no pacote `infrastructure/queue`. A camada de serviço injeta a interface, nunca a implementação direta — permitindo trocar o broker sem alterar lógica de negócio.
- **DT 3.1.17 Containerização:** A aplicação é empacotada como imagem **Docker**. O artefato Maven (JAR executável gerado pelo *spring-boot-maven-plugin*) é a base da imagem. Cada serviço roda em um container isolado.
- **DT 3.1.18 Orquestração:** O deploy é gerenciado por **Kubernetes (K8s)**. A aplicação é *stateless* (sem sessão HTTP), o que viabiliza escalabilidade horizontal com múltiplas réplicas sem coordenação de estado local — com exceção do cache de rate limiting, que exige backend distribuído (Redis) em ambientes multi-réplica.
- **DT 3.1.19 CI/CD:** O pipeline de integração e entrega contínua é implementado com **GitHub Actions**. O pipeline deve incluir, no mínimo: build Maven, execução de testes, build da imagem Docker e push para registry seguido de deploy no cluster Kubernetes.
- **DT 3.1.20 Banco Relacional: PostgreSQL** como banco de dados principal de produção. O acesso é feito exclusivamente via **Spring Data JPA** (*spring-boot-starter-data-jpa*) com Hibernate como provedor JPA. Entidades são mapeadas com anotações Jakarta Persistence (`@Entity`, `@Table`, `@OneToOne`, etc.).
- **DT 3.1.21 Geração de IDs:** Identificadores primários de entidades são gerados como **UUID** (`@GeneratedValue(strategy = GenerationType.UUID)`), armazenados como String. Isso garante IDs não sequenciais e globalmente únicos, adequados para ambientes distribuídos.
- **DT 3.1.22 Migrações de Schema:** O schema do banco é versionado e gerenciado por **Flyway** (*flyway-database-postgresql*). Toda alteração estrutural no banco deve ser realizada via script de migração versionado (`V{N}__descricao.sql`). A aplicação nunca modifica o schema automaticamente via `spring.jpa.hibernate.ddl-auto`.
- **DT 3.1.23 Cache e Armazenamento Temporário: Redis** (*spring-boot-starter-data-redis*) é utilizado para dois propósitos distintos: (1) cache de dados de alta frequência e curta duração; (2) armazenamento temporário de arquivos binários em trânsito. Redis **não** é utilizado como broker de mensagens - essa função cabe ao RabbitMQ.
- **DT 3.1.24 Banco em Testes: H2** (*in-memory*) como banco de dados em testes automatizados. O perfil de teste (`@ActiveProfiles("test")`) ativa as

configurações específicas para H2, garantindo isolamento total entre ambientes de teste e produção.

- **DT 3.1.25 Build e Dependências: Apache Maven** com arquivo `pom.xml` como sistema de build e gerenciamento de dependências. O empacotamento final é um JAR executável (*fat JAR*) gerado pelo *spring-boot-maven-plugin*, com todas as dependências embutidas.
- **DT 3.1.26 Configuração da Aplicação:** Configurações são gerenciadas via arquivos `.properties` (ou `.yaml`) do Spring Boot, com suporte a perfis (`spring.profiles.active`). Valores sensíveis (chaves RSA, tokens, credenciais) são injetados via variáveis de ambiente e nunca versionados no repositório. Propriedades customizadas agrupadas são tipadas via `@ConfigurationProperties` com validação pela Jakarta Bean Validation (`@Validated`).
- **DT 3.1.27 Profiles:** O sistema define ao menos dois profiles: **default** (produção, com PostgreSQL, Redis e RabbitMQ reais) e **test** (com H2 in-memory e dependências externas mockadas ou desabilitadas). Beans e configurações profile-específicos são anotados com `@Profile`.
- **DT 3.1.28 Documentação da API:** O sistema deve expor sua documentação técnica seguindo o padrão [OpenAPI 3.0](#) via **Swagger UI**. A geração deve ser automática a partir das anotações nos controllers, garantindo que o contrato da API esteja sempre sincronizado com o código.
- **DT 3.1.29 Estratégia de Versionamento:** A API adota o versionamento via **prefixo na URI** (ex: `/api/v1/...`). Mudanças que quebrem a compatibilidade com o cliente (breaking changes) exigem a criação de um novo conjunto de endpoints sob o prefixo subsequente, mantendo a versão anterior ativa durante o período de transição.
- **DT 3.1.30 Integridade de Payloads e Whitelisting:** O sistema deve rejeitar requisições com chaves duplicadas no JSON através da ativação do `STRICT_DUPLICATE_DETECTION` no Jackson ObjectMapper. Query strings duplicadas também devem ser rejeitadas via filtro de entrada. É obrigatório o uso de **DTOs** como *whitelist* explícita de campos; qualquer campo enviado no JSON que não esteja mapeado no DTO de entrada (ex: `is_admin`, `password_hash`) deve ser ignorado ou causar erro de deserialização.

## 3.2. Autenticação e Controle de Acesso

Detalhes técnicos da camada de autenticação do sistema, focando principalmente na segurança e implementação.

- **DT 3.2.1 Criptografia de Senha - Argon2id:** O sistema deve criptografar senhas em repouso utilizando o algoritmo **Argon2id**, superior ao Bcrypt em resistência a ataques de GPU/ASIC. Os parâmetros de custo devem ser fixados em: `time_cost = 3`, `memory_cost = 65536` (64MB) e `parallelism = 2`. O hash resultante é armazenado no campo `password` da entidade; a senha original nunca é persistida.

- **DT 3.2.2 Assinatura de Token - Algoritmo Assimétrico:** O sistema deve usar um par de chaves **RSA** (pública/privada) carregadas de arquivos PEM externos para assinar e verificar tokens **JWT**. O algoritmo resultante é **RS256** (RSA + SHA-256). A chave privada deve estar em formato PKCS#8 e a pública em formato X.509. Os caminhos dos arquivos são injetados via propriedades de configuração (`#{jwt.public.key}` e `#{jwt.private.key}`). A decodificação do JWT é feita com **NimbusJwtDecoder**.
- **DT 3.2.3 Estrutura do Token JWT:** O token **JWT** deve conter as seguintes claims:
  - **iss:** identificador da aplicação emissora.
  - **iat:** timestamp de emissão.
  - **exp:** timestamp de expiração.
  - **sub:** identificador único não-adevinhável do usuário (UUID), não o login/e-mail.
  - **jti:** UUID gerado a cada emissão.
  - **tenant\_id:** identificador do perfil de proprietário associado ao usuário (claim customizada).
- **DT 3.2.4 Revogação de Token e Refresh:** O sistema emite Access Tokens de curta duração (15 min) e **Refresh Tokens de longa duração (7 dias)**. O Refresh Token deve ser armazenado server-side (**Redis com TTL**) de forma **hasheada** e ser revogável individualmente no logout. O token deve incluir a claim **jti** para habilitar *blacklisting* e proteção contra *replay attacks*.
- **DT 3.2.5 Autenticação Stateless:** O sistema não deve criar ou utilizar sessões HTTP. A política de gerenciamento de sessão deve ser configurada como **STATELESS**, eliminando o uso de **HttpSession** pelo **Spring Security**.
- **DT 3.2.6 Desabilitação de CSRF:** Por ser uma API stateless autenticada exclusivamente por Bearer Token, a proteção CSRF deve ser explicitamente desabilitada. A ausência de cookies de sessão elimina o vetor de ataque que a proteção CSRF cobre.
- **DT 3.2.7 OAuth2 Resource Server:** O sistema deve ser configurado como um **OAuth2 Resource Server**, delegando a validação do Bearer Token ao filtro **BearerTokenAuthenticationFilter** nativo do **Spring Security**.
- **DT 3.2.8 Autorização:** O sistema deve usar **@EnableMethodSecurity(prePostEnabled = true)** para habilitar autorização em nível de método via **@PreAuthorize**. As regras por endpoint são configuradas no SecurityFilterChain via `.authorizeHttpRequests()`. Os endpoints POST `/auth/login` e POST `/auth/register` são os únicos *permitAll()*.
- **DT 3.2.9 Mapeamento de Roles:** As roles devem ser representadas como um enum persistido como String no banco de dados **@Enumerated(EnumType.STRING)**. A conversão para **GrantedAuthority** deve adicionar o prefixo **ROLE\_** ao nome do enum) via **SimpleGrantedAuthority**.
- **DT 3.2.10 User Authorities Filter:** Após a validação do JWT pelo **BearerTokenAuthenticationFilter**, um filtro customizado (**OncePerRequestFilter**) deve substituir o **Authentication** do



`SecurityContextHolder` por um novo objeto cujas autoridades são carregadas diretamente do banco de dados em cada requisição. Isso garante que as revogações de permissão sejam efetivas imediatamente, sem necessidade de emitir novo token. O `userId` é extraído da `claim sub` do **JWT** validado.

- **DT 3.2.11 Worker Auth Filter:** Requisições originadas de workers internos devem ser autenticadas via header HTTP customizado `X-Worker-Token`, cujo valor é comparado a um segredo compartilhado configurado via propriedade. Se válido, um `Authentication` com authority **ROLE\_WORKER** é inserido no `SecurityContextHolder`.
- **DT 3.2.12 Ordem dos Filtros de Segurança:** A ordem de execução dos filtros customizados é crítica e deve seguir a sequência: `RateLimitingFilter` → `BearerTokenAuthenticationFilter` → `WorkerAuthFilter` → `UserAuthoritiesFilter` → `AuthorizationFilter`. O sistema deve validar por testes de integração essa ordem na inicialização do servidor.
- **DT 3.2.13 Comparação de Segredos em Tempo Constante:** Qualquer comparação de tokens, chaves de API ou segredos deve usar comparação de bytes em tempo constante (e.g., `MessageDigest.isEqual()`) para prevenir timing attacks [1]. Nunca usar `.equals()` ou `==` para comparação de strings sensíveis.
- **DT 3.2.14 Headers de Segurança HTTP:** A configuração do `HttpSecurity` deve incluir explicitamente: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY` e `Strict-Transport-Security` com `includeSubDomains` e `maxAge` mínimo de 1 ano (em produção com HTTPS).
- **DT 3.2.15 Mensagens de Erro Genéricas e Resposta Furtiva:** Respostas de erro de autenticação devem retornar mensagens genéricas. Após o gatilho de *lockout* (conforme DT 3.2.23), o sistema deve retornar **HTTP 200 OK** com uma mensagem padrão de "sucesso na solicitação" ou "instruções enviadas por e-mail", sem confirmar que o usuário está bloqueado ou que a credencial era inválida, mitigando a enumeração de contas.
- **DT 3.2.16 Proxy Reverso de Borda (Cloudflare):** O sistema deve utilizar o Cloudflare como camada de proteção de borda. Ele é responsável pela terminação SSL/TLS, mitigação de ataques DDoS e aplicação de Rate Limiting volumétrico antes que a requisição atinja o servidor de origem.
- **DT 3.2.17 Integração Cloudflare e Precedência de Resolução:** O sistema deve priorizar o cabeçalho `CF-Connecting-IP` para identificação do cliente. Caso este cabeçalho esteja ausente, o sistema deve utilizar a estratégia **TRUSTED\_PROXY** (definida em DT 3.2.20) como primeiro fallback. Se nenhum proxy reverso for detectado ou configurado, o sistema deve obrigatoriamente utilizar a estratégia **DIRECT** (DT 3.2.20) para assegurar a integridade da origem.
- **DT 3.2.18 Fluxo Técnico de Recuperação de Senha:** A funcionalidade de reset de senha deve seguir o fluxo:
  - Geração de um UUID único (Token) com validade de 15 minutos armazenado no **Redis**.

- Postagem assíncrona dos dados (E-mail e Link) no **RabbitMQ**.
- Consumo e envio via API HTTP (**DT 3.3.1**).
- Validação de uso único (o token deve ser invalidado imediatamente após a troca bem-sucedida da senha).
- **DT 3.2.19 Proteção de Origem (Firewall)**: O servidor de aplicação deve ser configurado para aceitar conexões TCP exclusivamente dos intervalos de IP oficiais do **Cloudflare**, impedindo ataques que tentem "bypassar" o proxy reverso acessando o IP real da infraestrutura.
- **DT 3.2.20 Estratégias de Resolução de IP**: O sistema deve implementar dois modos de identificação conforme a infraestrutura de rede:
  - **DIRECT**: Obtém o IP diretamente da conexão TCP via `request.getRemoteAddr()`. É a abordagem padrão e mais segura para aplicações expostas diretamente à internet.
  - **TRUSTED\_PROXY**: Extrai o IP do cabeçalho `X-Forwarded-For`. Deve ser utilizada apenas atrás de proxies confiáveis, respeitando o parâmetro `trustedProxyDepth` para ignorar IPs forjados pelo cliente no início da lista.
- **DT 3.2.21 Rate Limiting Filter**: O filtro deve aplicar as regras de consumo de tokens baseando-se na identificação do cliente resolvida pelas estratégias definidas no **DT 3.2.20**. Deve operar em duas camadas:
  - **Camada Local (Bucket4j + Caffeine)**: Um limite estrito e de baixíssima latência via cache local (Caffeine) para barrar micro-bursts no mesmo milissegundo.
  - **Camada Distribuída (Bucket4j + Redis)**: O **Bucket4j** sincroniza o estado global; integrada ao **Redis** para armazenamento do estado dos tokens de forma distribuída

***Resultado:** Mesmo que a latência do Redis ocorra, a camada local impede que uma única instância processe mais do que o permitido em uma janela curtíssima, eliminando o efeito da race condition.*

- **DT 3.2.22 Transporte e Armazenamento de Tokens**: O **Access Token** deve ser entregue no corpo da resposta e armazenado pelo frontend **apenas em memória**, nunca em `localStorage` ou `sessionStorage`. O **Refresh Token** deve ser entregue exclusivamente via **Cookie** com as flags `HttpOnly`, `Secure` e `SameSite=Strict`, impedindo o acesso via JavaScript (XSS). No login, um par de tokens completamente novo deve ser gerado (*Session Fixation Protection*).
- **DT 3.2.23 Política de Lockout Progressivo**: O sistema deve implementar uma política de bloqueio progressivo baseada em sinais múltiplos (IP, Subnet, User-Agent):
  - **Login**: 5 tentativas em 15 min geram bloqueio de 15 min → 1H → 24h.
  - **Registro**: 3 tentativas em 1h geram bloqueio de IP/Subnet.
  - **Recuperação**: 3 tentativas em 1h; após isso, o sistema silencia e retorna sucesso falso (furtivo).



- **API Geral:** 300 requisições/min geram *Hard Block* de 1h.
- **DT 3.2.24 [Ocultação de Recursos e 404]:** Em endpoints de modificação ou acesso a recursos específicos, caso o usuário não possua a *ownership* (conforme DT 3.5.1), o sistema deve retornar **HTTP 404 Not Found** em vez de 403 Forbidden. Isso impede que atacantes confirmem a existência de IDs de terceiros através da diferença entre "Proibido" e "Não Encontrado".
- **DT 3.2.25 Prevenção de SSRF em Integrações:** Todas as URLs de saída para serviços externos (Resend, S3) e comunicação entre instâncias (Workers) devem ser validadas contra uma **Whitelist Estática**. É proibida a concatenação direta de inputs de usuários em URLs de requisição sem sanitização e validação de domínio.
- **DT 3.2.26 Limitador de Concorrência para Operações Intensivas:** O sistema deve implementar um semáforo ou um pool de threads dedicado e limitado para o processo de autenticação (Argon2id). O número de computações simultâneas de hash deve ser limitado a  $N$  (ex: **número de cores \* 1.5**), garantindo que a aplicação nunca utilize mais de 50% da memória disponível para hashes de login. Requisições excedentes devem receber **HTTP 503 Service Unavailable** imediatamente, protegendo o sistema contra exaustão de CPU/RAM.

### 3.3. Serviço de e-mail e notificação

Detalhes técnicos para envio de e-mails e notificações pelo sistema, com foco em desempenho, escalabilidade e simplicidade.

- **DT 3.3.1 Serviço de E-mail Transacional (API HTTP):** O sistema deve utilizar um provedor externo ([Resend](#)) para o envio de e-mails transacionais via API REST em vez de SMTP direto. O uso de SDK oficial deve ser priorizado para garantir resiliência, rastreabilidade (analytics) e facilidade de depuração.
- **DT 3.3.2 Mensageria Assíncrona (e-mail):** Toda tarefa de envio de e-mail deve ser processada de forma assíncrona para garantir a performance e a disponibilidade da API principal. A aplicação deve postar um payload JSON em uma fila do **RabbitMQ**, que será consumida por um Worker dedicado para realizar a integração com o provedor de e-mail via HTTP.

### 3.4. Observabilidade: Logs, Exceções, Testes Unitários e Métricas

Esta seção estabelece os padrões de telemetria, resiliência e qualidade para a operação do sistema em ambiente de nuvem.

- **DT 3.4.1 Estratégia de Logging e Sanitização:** O sistema deve emitir logs via **SLF4J** exclusivamente no console (**stdout**). Nenhum dado sensível (Senha, Token JWT, CPF ou CNPJ) deve ser logado em qualquer nível.
  - **TRACE:** Entrada de métodos com argumentos (habilitado apenas em desenvolvimento).

- **DEBUG:** Fluxos lógicos internos e decisões de filtros de segurança.
- **INFO:** Conclusão de casos de uso.
- **WARN:** Regras de negócio não atendidas e exceções de domínio (4xx).
- **ERROR:** Falhas críticas (5xx) e erros de integração externa, obrigatoriamente incluindo o *stack trace* completo .
- **DT 3.4.2 Tratamento Global de Exceções e Resposta de Erro:** As falhas devem ser capturadas por um `@ControllerAdvice` que as converte em objetos `ApiResponse<T>` padronizados .
  - **Hierarquia:** Exceções de negócio devem herdar de uma classe base (ex: `ApiBaseException`) que define o status HTTP.
  - **Segurança:** Erros genéricos (500) devem retornar mensagens opacas ("Internal Server Error") para evitar o vazamento de detalhes da infraestrutura .
- **DT 3.4.3 Distinção entre Falha de Autenticação e Autorização:** O sistema deve tratar de forma distinta a ausência de credenciais e a insuficiência de permissões.
  - **401 Unauthorized:** Retornado quando o token é ausente, inválido ou expirado.
  - **403 Forbidden:** Retornado quando o usuário está autenticado, mas seu `ROLE` não possui permissão para o recurso.

***Nota:** Falhas de autorização baseadas em propriedade de dados (ownership) devem seguir a regra de ocultação definida no DT 3.2.24 (404).*

- **DT 3.4.4 Métricas e Segurança do Actuator:** O monitoramento é realizado via **Spring Boot Actuator** integrado ao **Micrometer**.
  - **Endpoints:** Apenas `/health` e `/prometheus` devem ser expostos.
  - **Segurança:** O acesso deve ser restrito a uma porta de gerenciamento interna ou protegido por autenticação, nunca exposto diretamente na internet.
- **DT 3.4.5 Testes Unitários de Lógica de Negócio:** Métodos da camada de `service` com lógica condicional ou cálculos tributários devem possuir cobertura integral de testes unitários com **JUnit 5** e **Mockito**.
  - **Isolamento:** Dependências de `repository` e `spi` devem ser obrigatoriamente mockadas para validar exclusivamente a lógica da classe alvo.
- **DT 3.4.6 Testes de Camada Web (Slice Testing):** Controllers devem ser testados isoladamente via `@WebMvcTest` .
  - **Escopo:** Validar mapeamento de URL, códigos de status HTTP, serialização JSON e a correta aplicação das anotações de validação na entrada .
- **DT 3.4.7 Testes de Integração de Segurança:** A cadeia de filtros (JWT, Rate Limit, Roles) deve ser validada via `@SpringBootTest` com `MockMvc` .

- **Cenários:** Requisição sem token (401), Token com role inválida (403), Token expirado (401) e Tentativa de acesso a recurso de terceiros/Ownership (404).
- **DT 3.4.8 Monitoramento e Auditoria via AOP:** O sistema utiliza **Spring AOP** para automatizar preocupações transversais de forma desacoplada :
  - **Monitoramento de Performance:** Através da anotação `@LogExecutionTime`, o sistema registra o tempo de resposta de métodos específicos em nível `DEBUG`.
  - **Rastreabilidade de Auditoria:** Aspecto dedicado que intercepta operações de alteração de estado (criação, edição, deleção) para registrar o `userId` (via `SecurityContextHolder`), `timestamp` e a ação realizada. A imutabilidade é garantida pelo envio de logs estruturados (JSON) para o `stdout`, capturados por agregadores externos (`CloudWatch`) com políticas de retenção *append-only*.
- **DT 3.4.9 Sanitização de Entidades em Logs:** Entidades de domínio com dados pessoais devem sobrescrever o método `toString()` ou utilizar anotações de exclusão para impedir que dados sensíveis sejam impressos em logs.
- **DT 3.4.10 Formato Estruturado da ApiResponse para Erros de Validação:** O envelope `ApiResponse<T>` deve suportar um campo adicional opcional `errors: List<{field: String, message: String}>` para erros de validação (HTTP 400). Isso permite que os clientes consumam os erros por campo de forma programática, evitando o *parsing* manual de strings de texto.
- **DT 3.4.11 Cenários de Testes de Integração de Segurança:** O sistema deve ser validado (por testes de integração) contra os cenários mais prováveis de vulnerabilidade em SaaS (OWASP API Top 10):
  - **BOLA/IDOR (API1):** Teste de tentativa de acesso a um faturamento de outro `owner_id` via ID direto (esperado: 404).
  - **Mass Assignment (API3):** Tentativa de enviar `role: OWNER` ou `is_banned: false` em um endpoint de atualização de perfil (esperado: ignorado ou erro).
  - **Broken Auth (API2):** Script de brute-force para validar o *logout* progressivo e a resposta furtiva (200 OK).
  - **SSRF:** Teste de envio de URLs internas (127.0.0.1) em campos de URL de webhook ou integração.
  - **SQLi/XSS:** Injeção de scripts e queries em campos de busca e cadastro (validando sanitização do Hibernate e Jackson).
- **DT 3.4.12 Software Composition Analysis - SCA:** O pipeline de CI/CD deve incluir obrigatoriamente um passo de **SCA** (ex: Snyk, OWASP Dependency-Check). O build deve ser interrompido caso sejam detectadas dependências com vulnerabilidades de nível **HIGH** ou **CRITICAL** sem mitigação documentada.

### 3.5. Gestão, Isolamento e Precisão de Dados

Esta seção define as diretrizes para a integridade financeira, o isolamento entre clientes (*multi-tenancy*) e a persistência de documentos.

- **DT 3.5.1 Estratégia de Multi-Tenancy:** O isolamento de dados entre clientes é realizado através de uma **Discriminator Column**. Todas as tabelas com dados de negócio devem possuir obrigatoriamente a coluna `tenant_id` (vinculada ao UUID do usuário). O filtro de segurança deve ser aplicado de forma global na camada de persistência para impedir o acesso transversal entre utilizadores.
- **DT 3.5.2 Integridade de Transações e Consistência:** Todas as operações que alteram estado no banco de dados devem ser anotadas com `@Transactional`. Operações de leitura devem utilizar obrigatoriamente `@Transactional(readOnly = true)` para otimizar o *dirty checking* do Hibernate e reduzir *locks* no PostgreSQL. **Importante:** Transações nunca devem ser mantidas abertas durante chamadas a APIs externas (ex: BrasilAPI, Resend) para evitar a exaustão do pool de conexões; a transação deve ser aberta apenas para salvar o resultado.

## 4. Regras de Negócio (RNs)

| ID    | Regra                  | Descrição                                                                                    |
|-------|------------------------|----------------------------------------------------------------------------------------------|
| RN 01 | Atomicidade            | Toda persistência de dados deve ser transacional ( <i>all-or-nothing</i> ).                  |
| RN 02 | Validação: {tipo}      | O input deve seguir o padrão rigoroso do tipo especificado (Ex: e-mail, CPF, CNPJ).          |
| RN 03 | Fallback Manual        | Em caso de falha em serviço externo, habilitar campos para preenchimento manual.             |
| RN 04 | Log de Auditoria       | Registrar Timestamp, ID do Usuário, IP e Ação em log não editável.                           |
| RN 05 | Retenção Legal: {anos} | Os dados devem ser mantidos em modo <i>read-only</i> pelo período definido antes da deleção. |
| RN 06 | Bloqueio Lógico        | Impedir edição do campo após o primeiro registro de evento vinculado.                        |
| RN 07 | Anonimização           | Remover dados de identificação pessoal (LGPD), mantendo apenas dados agregados/fiscais.      |

|              |                                |                                                                                                         |
|--------------|--------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>RN 08</b> | <b>Rate Limit: {rpm}</b>       | Limitar número de requisições de forma configurável (RFs sem essa tag respeitam o Rate Limit padrão)    |
| <b>RN 09</b> | <b>Idempotência</b>            | Garantir que requisições idênticas enviadas em curto intervalo não geram registros duplicados no banco. |
| <b>RN 10</b> | <b>Versionamento de Termos</b> | Sempre que os Termos de Uso mudarem, o sistema deve exigir um novo aceite no próximo login do usuário.  |